

Using Params, Request & Forms In A Controller

We already know how to work with params and the request object within a route. We can also use them in a controller. Let's see how we can do that. We are also going to create a form in a view and process the form in a controller.

Using Params In A Controller

We can use params in a controller the same way that we did in a route. We can type-hint the params in the method signature. Let's add a new method to our `JobController` that will take a param. We will create a method called `show` that will take a `jobId` param. Here is what the `JobController` class should look like:

```
class JobController extends Controller
{
    //...

    // Show job details
    public function show($id)
    {
        return "Showing job $id";
    }
}
```

Now, let's create a new route that uses the `show` method of the `JobController`. Open the `routes/web.php` file and create a new route that uses the `show` method of the `JobController`:

```
Route::get('/jobs/{id}', [JobController::class, 'show']);
```

You should see the string `Showing job {id}` when you visit `/jobs/1` in your browser. You can pass the `jobId` param to the `show` method and use it in the method.

Using The Request Object In A Controller

We can also use the request object in a controller. A common use of the request object is to get the input from a form. So let's create two methods in our `JobController` that will show a form and process the form. Here is what the `JobController` class should look like:

```
class JobController extends Controller
{
    //...

    // Show the form to create a job
    public function create()
    {

```

```
        return view('jobs.create');
    }

    // Store a job
    public function store(Request $request)
    {
        return $request->all();
    }
}
```

Create The Form View

Create a file at `resources/views/jobs/create.blade.php` and add the following code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <metaname="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Create Job</title>
  </head>

  <body>
    <h1>Create Job</h1>
    <form action="/jobs" method="POST">
      @csrf
      <input type="text" name="title" placeholder="Title" />
      <input type="text" name="description" placeholder="Description" />
      <button type="submit">Submit</button>
    </form>
  </body>
</html>
```

@csrf

You may have noticed the `@csrf` directive in the form. This is a security feature in Laravel to prevent cross-site request forgery (CSRF) attacks. It generates a hidden input field with a CSRF token that Laravel uses to verify the form submission. This will stop malicious users from submitting forms to your application without your knowledge. This is another reason why Laravel is so secure out of the box.

Using The Controller In A Route

Now that we have created our controller and added methods to it, we can use it in routes. Open the `routes/web.php` file and update the routes to use the `JobController`:

```
Route::get('/jobs/create', [JobController::class, 'create']);
Route::post('/jobs', [JobController::class, 'store']);
```

Make sure that you use the `post` method in the form and the route to process the form. Now when you visit `/jobs/create` in your browser, you should see the job form. When you submit the form, you should see the form data dumped to the screen.

You can also get individual form fields using the `input` method of the request object. Here is how you can get the `title` and `description` fields from the request object:

```
public function store(Request $request)
{
    $title = $request->input('title');
    $description = $request->input('description');

    return "Title: $title, Description: $description";
}
```

Route Order

The order is important here because we have a route that uses the `show` method of the `JobController` that matches the `/jobs/{id}` route. If you put the `/jobs/{id}` route before the `/jobs/create` and `/jobs` routes, the `show` method will be called instead of the `create` and `store` methods. This is because Laravel will match the first route that it finds. So make sure that the `/jobs/{id}` route is at the bottom of the file.

Now go to `/jobs/create` in your browser and create a job. You should see the form data dumped to the screen. This will include the name, description and the CSRF token. You can use this data to save the job to the database or do whatever you want with it.